

Article

Building a Self-Explanatory Social Robot on the Basis of an Explanation-Oriented Runtime Knowledge Model

José Galeas ¹, Alberto Tudela ¹, Óscar Pons ¹, Suna Bensch ², Thomas Hellström ²
and Antonio Bandera ^{1,*}

¹ Departamento Tecnología Electrónica, University of Málaga, 29071 Málaga, Spain; jgaleas@uma.es (J.G.); ajtudela@uma.es (A.T.); opfernandez@uma.es (Ó.P.)

² Department Computing Science, University of Umeå, 901 87 Umeå, Sweden; suna@cs.umu.se (S.B.); thomash@cs.umu.se (T.H.)

* Correspondence: ajbandera@uma.es

Abstract

In recent years, there has been growing interest in developing robots capable of explaining their behavior, thereby improving their acceptance by humans with whom they share their environment. Proposed software designs are typically based on the advances being made in conversational systems built on deep learning techniques. However, apart from the ability to formulate explanations, the robot also needs an internal episodic memory, where it stores information from the continuous stream of experiences. Most previous proposals are designed to deal with short streams of episodic data (several minutes long). With the aim of managing larger experiences, we propose in this work a high-level episodic memory, where relevant events are abstracted to natural language concepts. The proposed framework is intimately linked to a software architecture in which the explanations, whether externalized or not, are shaped internally in a collaborative process involving the task-oriented software agents that make up the architecture. The core of this process is a runtime knowledge model, employed as working memory whose evolution allows for capturing the causal events stored in the episodic memory. We present several use cases that illustrate how the suggested framework allows an autonomous robot to generate correct and relevant explanations of its actions and behavior.

Keywords: explainability; robotics cognitive architecture; eXplainable Autonomous Robot



Academic Editors: Verónica Vasconcelos and Maryam Abbasi

Received: 7 July 2025

Revised: 30 July 2025

Accepted: 5 August 2025

Published: 10 August 2025

Citation: Galeas, J.; Tudela, A.; Pons, Ó.; Bensch, S.; Hellström, T.; Bandera, A. Building a Self-Explanatory Social Robot on the Basis of an Explanation-Oriented Runtime Knowledge Model. *Electronics* **2025**, *14*, 3178. <https://doi.org/10.3390/electronics14163178>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

As shown in Figure 1, it is increasingly common today to find service robots performing jobs (e.g., transporting goods in warehouse logistics, delivery in hospitals or offices, or as waiters) where they share the environment with humans, with whom, in many cases, some kind of interaction is established. When the robot has the tools to carry out verbal interaction, trust in robots can be increased by having them generate explanations that allow the person to know why they are doing what they are doing [1]. In general, it is about the robot being able to explain the reasons behind a certain decision, which are often hidden by the adoption of complex control policies, in many cases based on machine learning techniques. This is what is known as eXplainable Autonomous Robots (XARs) [2], and it does not only concern verbal explanations, but may also be accompanied by a behavior that, in general, offers strong assurance [3]. As has been highlighted in numerous papers, the difference between XAR and eXplainable Artificial Intelligence (XAI) lies mainly in the

fact that the robot is an embodied agent, which performs a certain action in the surrounding environment. While XAI typically focuses on justifications of output from decision-making systems (data-driven explainability), XAR focuses more on explaining actions or behavior of a physical robot (goal-driven explainability) to an interacting human [2].

In order to generate an explanation, the robot must have the mechanisms that allow it to handle both information about past experiences and factual knowledge. This knowledge is organized, respectively, in so-called episodic memory and semantic memory. Semantic memory organizes the concrete and objective knowledge that the robot possesses about a specific topic (general concepts, relationships, and facts abstracted from particular instances). Episodic memory is a type of long-term memory that allows the robot to recall past experiences, such as specific events and situations, including details about where and when they occurred. Both constitute conscious or declarative memory [4]. Using these declarative memories, the explanation will be constructed using both definitions or concepts and a temporal sequence of actions, performed or considered [5]. It is also necessary that all this knowledge model is shared with the humans with whom it is going to communicate; otherwise, neither party will understand the other [6]. Specifically, episodic memory is crucial to enable the robot to verbalize its own experiences. The representation used to store this information must synthesize the continuous flow of experience, organizing it and nimbly retrieving, when necessary, important past events to construct a response to the person [7].



Figure 1. Service robots examples: (left) the Cary up robot for warehouse logistics and (right) the Morphia robot in the Vitalia retirement home (Malaga).

Early work on verbalisation of episodic memory relied on rule-based verbalisation of log files or on fitting deep models on either hand-created or self-generated datasets. For instance, Rosenthal et al. [8] propose a verbalization space that manages all the variability of utterances that the robot may use to narrate its experience to a person. Garcia-Flores et al. [9] describe a framework for enabling a robot to generate narratives composed of its holistic perception of a task performed. Bärman et al. [10] propose a data-driven method employing deep neural architectures for both creating episodic memory from previous experiences and answering in natural language questions related with such experience. This requires the collection of a large multi-modal dataset of robot experiences, consisting of symbolic and sub-symbolic information (egocentric images from the robot's cameras, configuration of the robot, estimated robot position, all detected objects and their estimated poses, detected human poses, current executed action, and task of the robot). The problem with these approaches is that they require either the design of a large number of rules or the collection of large amounts of experience data. To avoid these problems, language-based representations of past experience can be used, which could be obtained from pre-trained multimodal models. In that case, one can pass the question, and the history as text, to a Large Language Model (LLM) [11] and ask it to generate the answer. The problem with this

approach is that, since episodic memories can store a lot of detail, this scheme only works well for short stories (although LLMs can process more and more tokens, it is important to keep this number limited in order to reduce response time). Furthermore, the correctness of such LLM-generated answers cannot be guaranteed, which is highly unacceptable in many applications [12].

To scale the generation of explanations to life-long experience streams, while maintaining a low token budget, we propose to derive a causal log representation from the evolution of the internal working memory. In our case, this working memory is the one used in the CORTEX cognitive architecture for robotics [13]. The information stored in this working memory is represented using linguistic terms, and is shared by all the software agents that make up the internal architecture endowed in the robot. The evolution of the working memory describes the robot's knowledge of the environment, including not only perceptions, but also actions or intentions [13]. It is a representation that, for a given instant, could be passed to an LLM to obtain a verbal description of the context, implementing a kind of soliloquy or inner speech [14]. In this case, this working memory representation facilitates the creation of the aforementioned causal log, a high-level episodic memory.

Figure 2 provides an overview of how our system works. Briefly, it processes the continuous stream of experiences encoded on the working memory to extract relevant snapshots, which are inserted into a causal log, our high-level episodic memory. These snapshots are identified at design time, and basically, they coincide with the preconditions that launch a specific action of the robot (detect a person, a low battery value, etc.). They are represented as natural language concepts. When the robot needs to provide an explanation, it uses this causal log, as well as semantic memory, to generate a raw explanation that is refined using the Large Language Model Meta AI (LLaMA) (<https://www.llama.com/>, accessed on 2 July 2025).

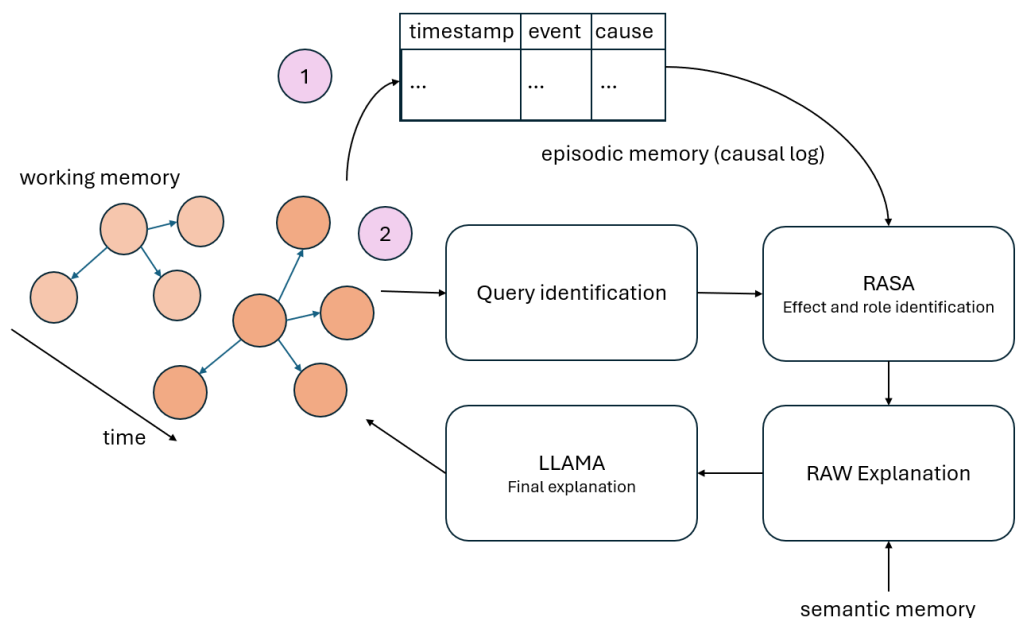


Figure 2. An overview of the approach: (1) The episodic memory (causal log) is updated with events captured from the working memory; and (2) given a question, the causal log information, the semantic memory, and the question are employed to generate a raw explanation. Syntactical refinement and LLaMA are employed to generate the final explanation.

1.1. Contributions

The idea of representing episodic memory using natural language in order to combine information from this memory with the question itself and, using deep learning models

(LLMs), generate the answer is not new. The problem is therefore how to represent the information in this memory and how to use this information to generate explanations for specific questions.

In our previous work, we proposed CORTEX, a software architecture for robotics in which all functionalities (perceptual, action, decision making, etc.) are organized as agents that communicate with each other by updating a runtime knowledge model [13]. This model represents a working memory, organized as a directional graph, in which nodes and arcs store semantic content. Thus, for a given instant of time, this memory could be passed to an LLM and would generate a text expressing what the robot internalizes (e.g., “The battery level is very low, so the current action has been aborted and I am heading to the charging station.”). While the temporal evolution of this working memory would generate a comprehensive episodic memory, it would be difficult to use it to generate an explanation covering a relatively long period of time.

In the deployment of CORTEX described in [15], the robot’s behavior was encoded in a set of Behavior Trees, each one associated with a specific task, and among them, a self-adaptation mechanism made it possible to determine which one was active depending on the context. These Behavior Trees are composed of more atomic actions, which require pre-conditions (person detected, menu not previously asked, etc.) for their execution. These items, which are relevant for the behaviors to take place, will constitute the so-called causal events. These events, which are captured from working memory, will constitute our episodic or causal log memory, characterised by its relatively high level of abstraction.

Alongside this memory, this article proposes a structure of agents that, connected to the working memory already referred to, determines when an explanation should be generated, shapes it, and manages its verbalisation. This pipeline uses deep learning computational models in several steps.

To summarize, we present a full integration of human–robot interaction components forming a unified robot architecture CORTEX, where the integrated components include robot perception, natural language usage, and robot action execution.

1.2. Organization of the Paper

The rest of the paper is organized as follows: Section 2 briefly introduces some technical preliminaries and describes the generation of explanations in a scenario where the robot tries to explain to an interacting person the reason for the actions it has executed or is executing. That is, it does not need to revisit the plan to determine, for instance, what happens if, instead of one action, another one would have been executed. The focus is on describing which knowledge representations the robot uses to generate these explanations. The software architecture CORTEX and the runtime knowledge model, the Deep State Representation (DSR), are briefly described in Section 3. The scheme proposed for building a high-level, abstracted episodic memory is presented in Section 4. Section 5 describes the software framework designed for generating the explanations. Both episodic memory and explanation generation have been integrated into CORTEX, allowing, for example, memory to be continuously updated or explanations to be generated at any time during the execution of a plan. Section 6 details the generation of a causal log and the explanations provided to several questions. In Section 7, relevant issues are discussed and future lines of research are proposed.

2. Technical Preliminaries and Related Work

There are several papers describing representation schemes used as a basis for enabling a robot to generate explanations of its past behavior [2,16]. This behavior can be expressed as the set of actions that transformed an initial state s_0 into a current state s_n . In this

sequence, each state $s_i \in S$ can be described by a set of propositional variables $v_i \in V$. Each robot action $a_i \in A$ has preconditions $pre(a_i)$ and effects $eff(a_i)$, which can be also characterized by these same variables [17]. Thus, an action a_i is applicable in a state s_i if its preconditions are true ($pre(a_i) \subset s_i$). In the same way, when a_i is applied, the state s_i evolves to s_{i+1} , and the set of variables v_i is updated to satisfy $eff(a_i)$.

Using this terminology and denoting the decision-making space with Π , the plan $\pi \in \Pi$ is a sequence of actions [2], as follows:

$$\pi = \{a_1, a_2, \dots, a_i\}, \quad a_i \in A, \quad (1)$$

which is generated using an algorithm $\mathcal{A}$ and under a certain constraint τ , i.e.,

$$\mathcal{A} : \Pi \times \tau \rightarrow \pi, \quad (2)$$

A causal link between two consecutive actions, (a_i, e, a_{i+1}) , implies that there exists a set of facts e that are part of the effects of a_i , $e \subset eff(a_i)$, and of the preconditions of a_{i+1} , $e \subset pre(a_{i+1})$ [17].

The goal of an explanation is to justify to the person that the solution π satisfies τ for a given Π (or that a wrong state has been unexpectedly reached and the plan was stopped [18]). Assuming that the person is intimately familiar with the internal model of the robot, with how it organizes knowledge, and with how the planner decomposes the execution of a plan, the verbalisation of a plan or previous experience consists of a step-by-step plan explanation (considering states and actions) to the person [6,17]. Interpreting causal links as models of causal relationships between actions, Stulp et al. [19] propose to build an ordered graph for the plan that can help the robot to answer *why* questions. Nodes of this graph are actions of the plan, and links are composed of variables that fulfill

$$\exists v_x \in eff(a_i) | v_x \in pre(a_j), i < j \quad (3)$$

Advancing backwards in the plan, every precondition of an action is linked to the latest previous action that created it. Thus, the effect of a given action can be linked to the precondition of one or more other actions. Each precondition links to at most one effect [19].

Additionally, using the concept of causal link, Canal et al. [20,21] propose PlanVerb, an approach for the verbalisation of task plans based on semantic tagging of actions and predicates. This verbalisation is summarized by compressing the actions acting on the same variables or facts. They consider four levels of abstraction for plan verbalisation. In the lowest level, the verbalisation considers numerical values (e.g., real-world coordinates of objects, time duration of actions). In the highest one, only the essential variables of the actions are verbalised.

The episodic H-EMV memory, proposed by Bärman et al. [7], is also organized as a hierarchical representation, in which information is organised into raw data, scene graph, events, goals, and summary. Each level builds on the previous level, and the nodes that form it can have children of different types (e.g., a goal node can have linked events or other goal nodes; this allows subgoals of complex tasks to be represented). The H-EMV can be interactively accessed by an LLM to provide explanations or answers to questions from the person, keeping the list of tokens bounded even when the explanation covers a large period of time.

Finally, Lindner and Olz [17] discuss in depth the problem of constructing explanations purely on the basis of causal links. The problem is that, in many cases, an action is not necessarily executed to achieve an initial goal, but may arise to fulfill a partial goal that appears during the execution of the plan (even originating from the execution of the plan

itself). There are also occasions when two actions are not linked by preconditions or effects, but there is a logical causal relationship, underlying the execution of the plan itself. With the aim of distinguishing effects that render subsequent actions necessary from effects rendering subsequent actions possible, they define semantic roles for the effects. Thus, an effect is defined as a *demander* one when the goal could have been reached without considering it, but it makes an additional action necessary. For example, imagine a robot apologizing (*SaySorry*) to a person for passing too close to it. Apologizing is not necessary to achieve the ultimate goal of reaching a certain location, and has been launched as a consequence of an effect (passing too close to the person). This effect is the demander of the action *SaySorry*. Preconditions that are not demanders are defined as *proper enablers*. It is important to note that an effect can play both roles at the same time. By distinguishing between links associated with demanders and enablers, the authors show how the robot is able to generate better explanations from the person's point of view.

3. CORTEX

The CORTEX (<https://github.com/grupo-avispa/cortex>, accessed on 8 August 2025) robotic architecture is a mature approach, which has been used in different projects and platforms [13,15]. Basically, the idea is to use a runtime knowledge model as a working memory that builds the software agents in which the robot's functionality is structured. All the information needed by the agents is in this memory, allowing deliberate or reactive behaviors to emerge. Figure 3 shows what the instantiated architecture looks like in the examples that will be described in this article.

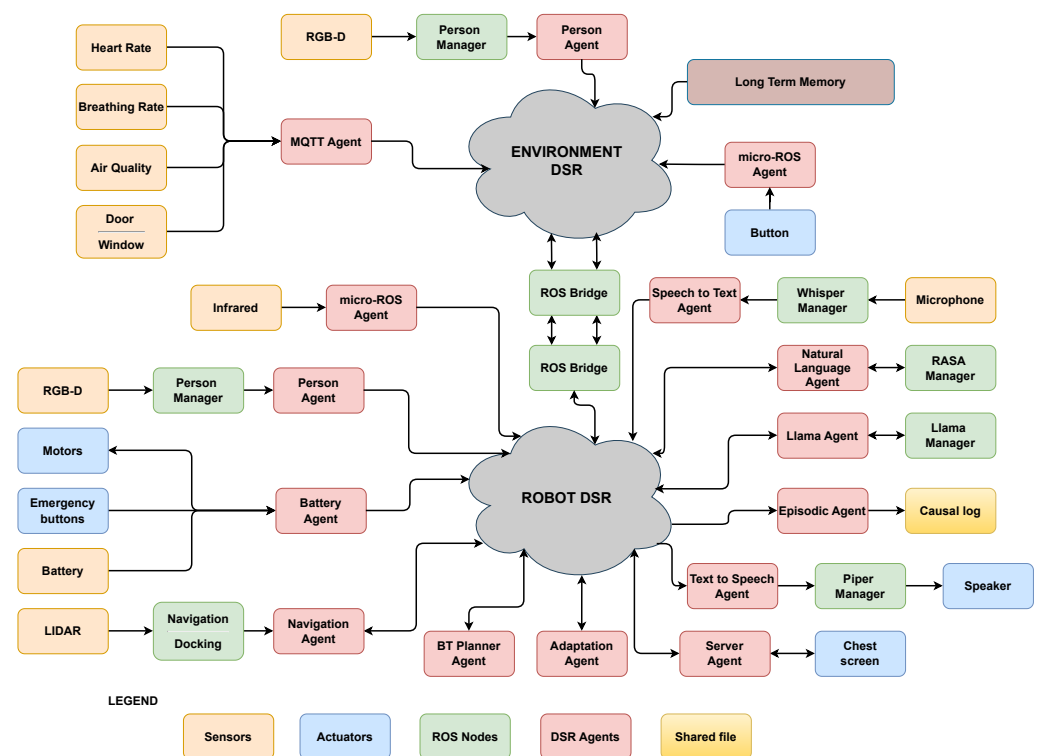


Figure 3. Instantiation of the CORTEX architecture used in the use cases described in this paper. The blocks encapsulate embedded software to control sensors and actuators, ROS nodes, software agents connected to the DSR, and shared files. The type of block is marked by the fill colour (see legend in the bottom row of the figure).

Following the scheme proposed in a previous work [15], the model can be extended to consist of two independent but connected working memories: one internal to the robot

itself and the other associated with the environment. Each one can maintain its own functionalities (software agents) and its own associated hardware. The important point is that, in these working memories, most of the information is annotated semantically. As an example, Figure 4 shows the evolution of the working memory in the robot when it detects that the person it was following has disappeared. The graph shows a reduced version of the working memory on the robot, where only the most significant nodes and arcs have been left for clarity. If the content of this memory is analyzed, it is easy to note down in sentences what is happening at each instant of time (“robot is with the therapist”, “robot is moving and following the person”, etc.). This kind of inner speech [22] serves as a basis for the construction of a high-level episodic memory and thus for our proposal for the generation of explanations.

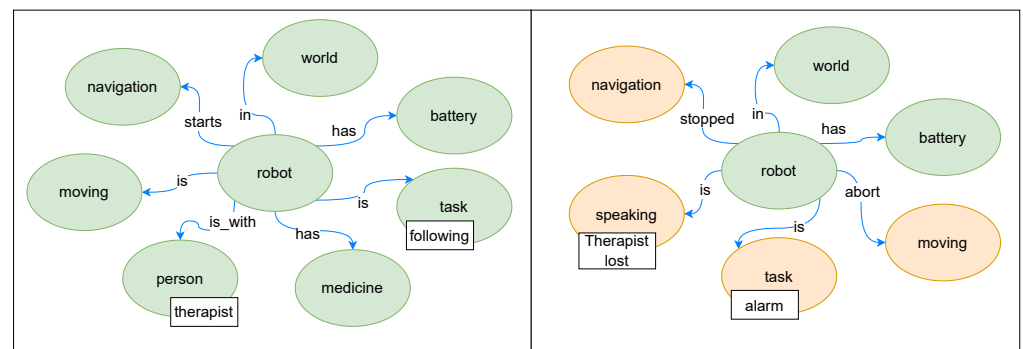


Figure 4. DSR evolution when the robot lost the person it was following.

The Deep State Representation (DSR)

As aforementioned, a working memory, the so-called Deep State Representation (DSR), is at the core of each one of the instantiations of CORTEX shown in Figure 3. The DSR is a distributed, directed, multi-labeled graph, which acts as a runtime model [15]. The graph nodes are elements of a predefined ontology, and they can store attributes such as raw sensor data. On the other hand, the graph arcs are of two different types. Geometric arcs encode geometric transformations between positions (e.g., the position of a robot's camera with respect to the robot's base). These relative probabilistic poses between graph nodes are represented as rotation and translation vectors with their covariance. Symbolic arcs typically encode logic predicates. Thus, for example, the graph could maintain the robot's position with respect to the current room and all elements relevant to an ongoing interaction with a person. Graph arcs can also store a list of attributes of a predefined type.

In the evolution of the graph in Figure 4, only the most significant symbolic arcs have been shown. Initially, the robot is following the therapist, carrying a medicine that the therapist needs. At a given moment, for whatever reason, the robot loses the person it is following. The ongoing task (*Following*) is aborted, and the robot's behavior is now guided by a new one (*Alarm*). The robot stops and asks the therapist to come to it to restart *Following*. The evolution of the DSR is a joint action of the software actors in CORTEX. The communication between them is through changes in nodes and arcs of the graph, and since the information in these is mostly semantic, this interaction can be interpreted as a conversation. Being internal, the result is an inner speech. Figure 5 shows the agents involved in the DSR evolution shown above. It is important to note that there are no messages between agents or from a central agent acting as Executive. The agents note the information in the DSR (e.g., the person is lost), and the system acts (e.g., the self-adapting agent changes the behavior from *Following* to *Alarm*).

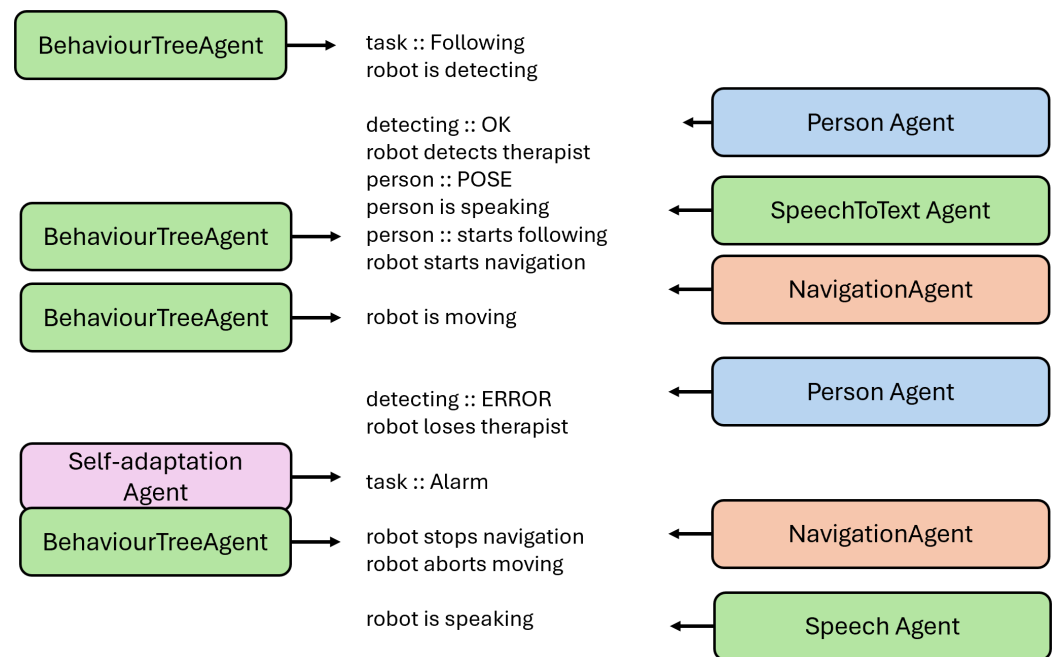


Figure 5. Example of inner dialogue in the DSR (see text).

4. Building a High-Level Episodic Memory

4.1. Semantic Memory

All information that different agents can annotate as nodes in the DSR has an associated semantic definition in a memory. This memory is available to any software agent in CORTEX, but it will be especially useful for the agent responsible for generating the explanations to construct the sentences with which to interact with humans. Table 1 shows examples of definitions stored in this memory, associated with robot events.

Table 1. Some examples of semantic definition in memory.

Robot Event	Description
Robot.wake_up = true	The robot was turned on
Robot.battery = low	The robot's battery is running low
Person_detected(therapist)	The therapist was detected by the robot's camera
Speech.command = follow	The therapist commands me to start following him
Robot.task = following	The robot started the use case for following the therapist

4.2. High-Level Episodic Memory

As in Bärmann's proposal [7], our implementation of episodic memory maintains a hierarchical structure. However, it has only two levels, of which only the top level is used for the generation of explanations. At the lowest level of this episodic memory, a record is kept of all changes that take place, at the node or arc level.

However, in this article, we are interested in high-level episodic memory, generated by a specific software agent, which detects certain situations and records them in a so-called causal log [14]. Table 2 represents some of the events and causes that can take place during an execution of the *Following* task. The example links to the evolution of the DSR shown in Figure 4 and its deployment as an internal conversation (Figure 5). In this example, the robot starts to follow the therapist once he has been detected and he commands it. Finally, at some point during the execution, the robot loses sight of the therapist and aborts the task.

Table 2. The robot records all occurrences of causal events in the causal log that is represented as a table. The causal log contains causal chains or causal dependencies of linked causal events.

	Timestamp	Event	Cause
1	1727677625	Robot.wake_up = true	-
2	1727677626	Robot.task = explore	Robot.wake_up = true
3	...	...	...
4	1727679120	Person_detected(therapist)	Is_in_front_of_robot $\wedge$ Lights_on
5	1727679122	Robot.task = interaction	Person_detected(therapist)
6	...	...	...
7	1727679130	Speech.command = follow	Speech_recognition = follow
8	1727679133	Robot.task = following	Speech.command = follow
9	...	...	...
10	1727679252	Robot.task_abort = following	Robot.lost(therapist)
11	1727679252	Robot.task = alarm	Robot.task_abort = following
12	...	...	...

High-level episodic memory is thus constructed as a table in which certain events are linked. These robot events are associated with states/actions in the Behavior Trees that control the behavior of the robot. As shown in Figure 5, the evolution of the DSR over time enables these significant events to be extracted. These events represent either specific steps of the use case in progress (encoded at design time in the Behavior Trees that control the robot's behavior) or situations that have precipitated the change in behavior, aborting a given Behavior Tree and launching a new one. As proposed in [17], there are enablers, which lead to the completion of the current task, but the system supports the concept of demanders. In fact, when the person is lost, a new behavior is triggered, and the episodic memory records this change and the evolution of this new behavior. The logic underlying this change prevents the explanation from giving this new behavior an intentional character (it has not been sought), avoiding the problem described by Lindner and Olz [17].

The online construction of this episodic memory is intimately linked to the robot's action planning scheme. However, it is not a mere collection of preconditions, actions, and effects. They are significant milestones, which allow episodic memory to have a high level of abstraction, being suitable for handling explanations related to long-term experiences. The following subsection provides a brief discussion of how planning is managed, emphasizing its relationship to episodic memory management.

4.3. Planning and Self-Adaptation Based on Behavior Trees

Figure 6 shows the Behavior Tree that manages the *Following* task using the Groot2 (version 4.6) tool (<https://github.com/BehaviorTree/Groot2>, accessed on 23 June 2025). The code includes what is needed to be able to be aborted by the self-adapting agent. Briefly, it consists of a loop in which the position of the person is detected and the robot is moved to a position close to it. The task is successfully completed when the person asks the robot to stop. Most of the events shown in Table 2 are present in this Behavior Tree (Person_detected, Speech.command). Others are associated with ongoing or changed behavior (Robot.task, Robot.task_abort).

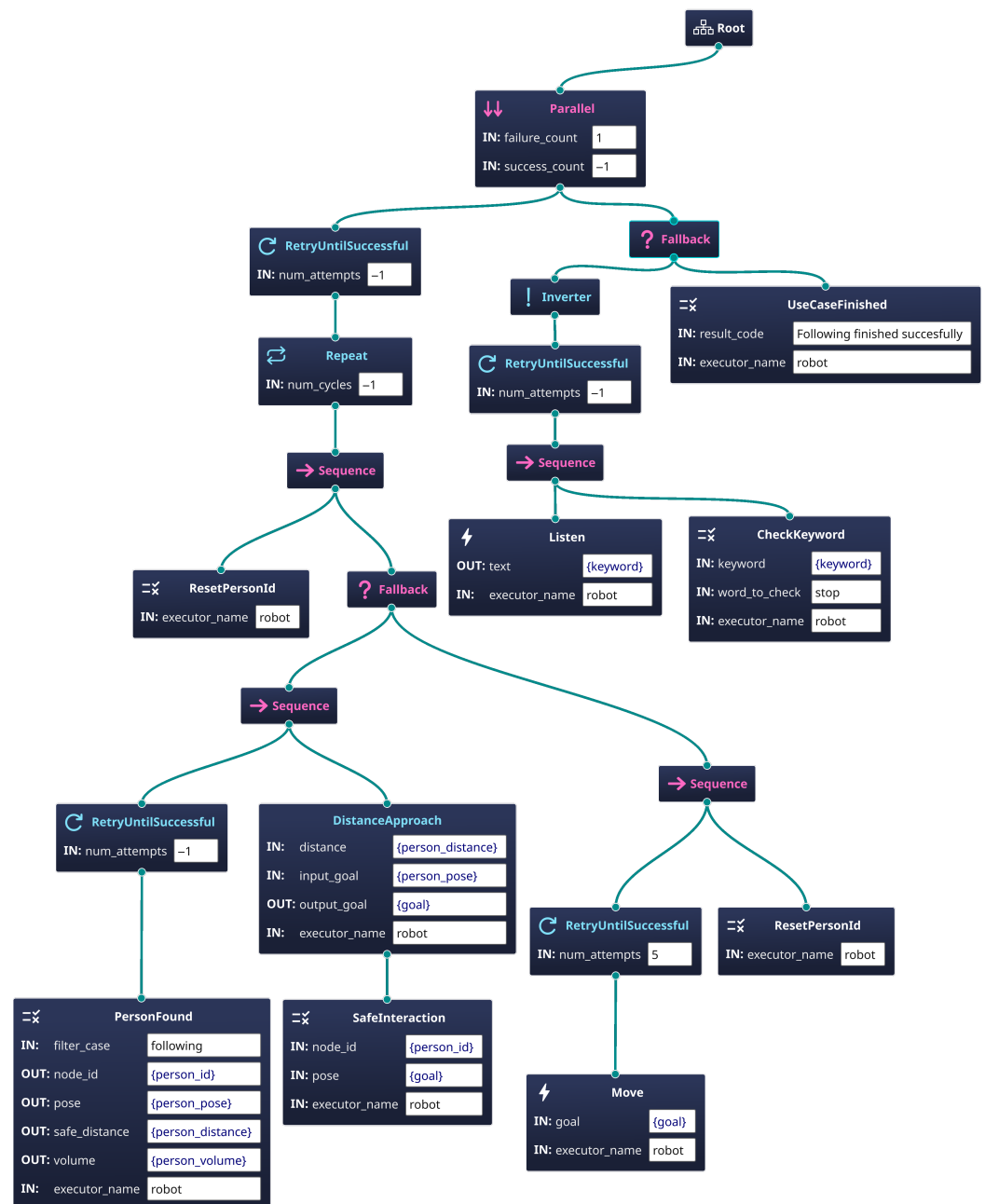


Figure 6. Following Behavior Tree.

The self-adapting agent is in charge of selecting the behavior for the robot that best fits the current context. This is achieved through the continuous estimation of specific metrics [22]. These metrics determine which behavior the robot should adopt, either by letting the current behavior continue or by aborting it when a new behavior should emerge (e.g., deciding that the robot should go to the charging station because its battery level will prevent it from successfully completing the current task). In the current situation (a robot waiting for a caregiver or therapist to place a certain medicine on its tray and ask it to follow), the default behavior is *Interaction*, where the robot is waiting for commands. Once the 'following' command is received, the robot starts executing the *Following* behavior, as explained in the Behavior Tree in Figure 6. To continue with the *Following* execution and not to abort this behavior, it needs to be checked that (i) the mission is going well (the medicine is on the robot's tray and the distance to the person is correct), (ii) there is power autonomy, and (iii) the safety of the path is ensured. All of these situations can be

concretized in metrics, which encode certain variables captured from the context, internal or external to the robot [22].

5. Generating the Explanation

Faced with a given request (question) from the person, the robot uses episodic and semantic memories to prepare an answer (explanation). Figure 7 shows the Behavior Tree that manages this task. Essentially, the robot detects the question and estimates the role of the person asking it (i.e., whether they are a therapist, robot technician, or resident, etc.) [14]. Identifying the role allows for personalizing the explanation. With this information, a first version of the explanation is built, which will be refined using a specific software agent (LLaMA agent). The final explanation is verbalised by the robot, but is also displayed on the screen on the robot's torso.

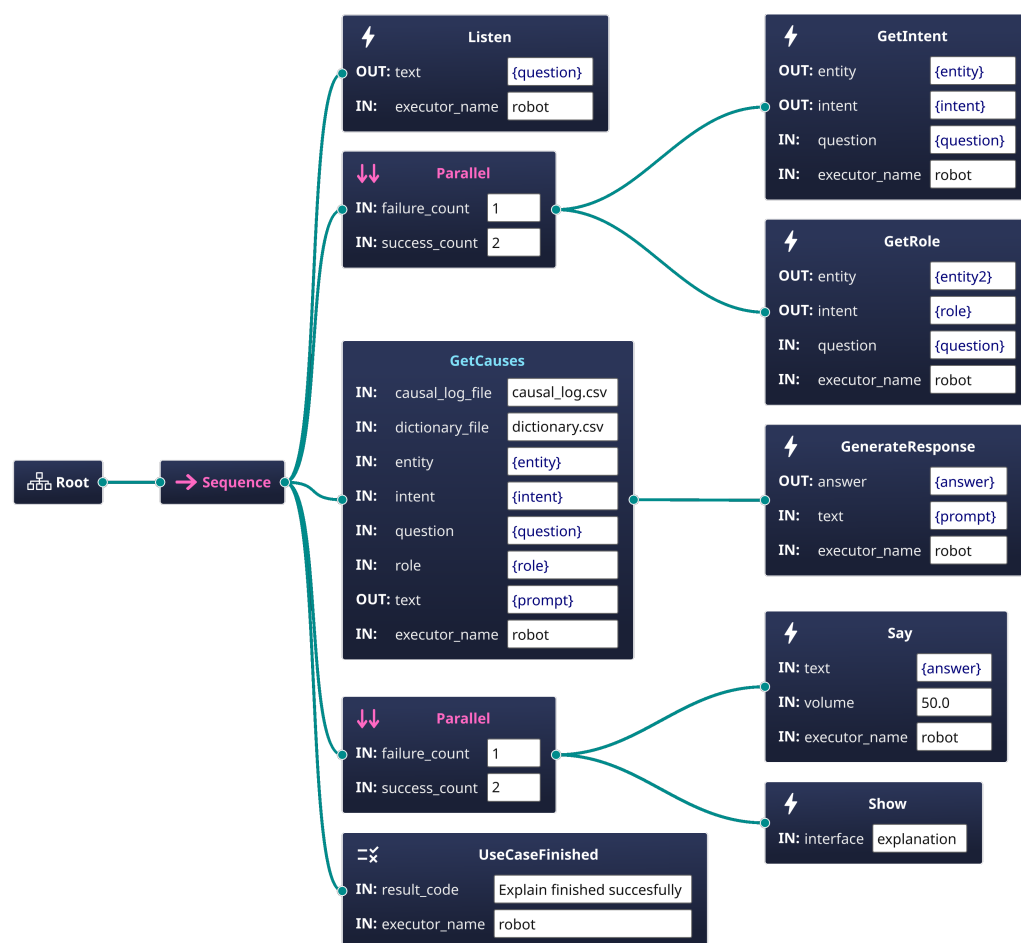


Figure 7. Explanation Behavior Tree.

The software agents involved in this task are shown in Figure 8. A Speech To Text (STT) agent, responsible for capturing what the person says and identifying whether it is a question, is deployed in CORTEX. The questions are passed to a second Natural Language Processing (NLP) agent. This agent is responsible for detecting which robot event the question refers to and identifying the role of the person asking the question. With this information, and using the data in the semantic and episodic memories, a first raw version of the explanation is constructed. This explanation is not very human friendly, so it is refined using the LLaMA agent. This text is verbalized (Text To Speech (TTS) agent) and displayed on the screen on the torso of the robot (Web agent). Further details on these agents are provided below.

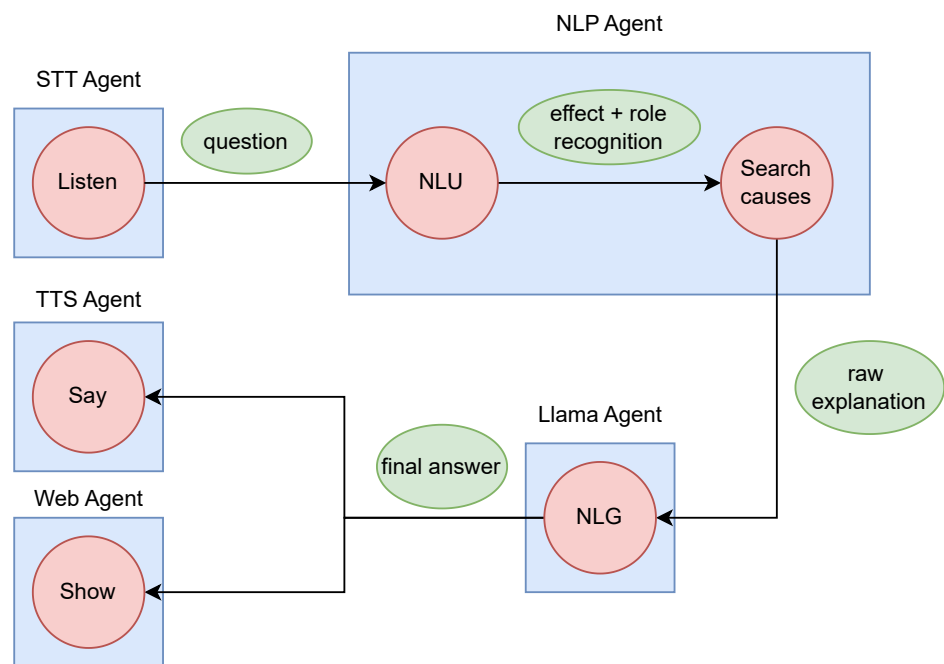


Figure 8. Software agents involved in the generation of an explanation (see text).

5.1. Speech To Text Agent

The Speech To Text (STT) agent is in charge of executing the whole process to solve the problem of transcribing what is heard into text. It is designed to facilitate its integration in the CORTEX architecture, being connected to the DSR network. Basically, it is in charge of detecting the audio, transcribing it to text, and uploading it to the DSR as an attribute. For this, the whole process is based on the ROS 2 pipeline shown in Figure 9.

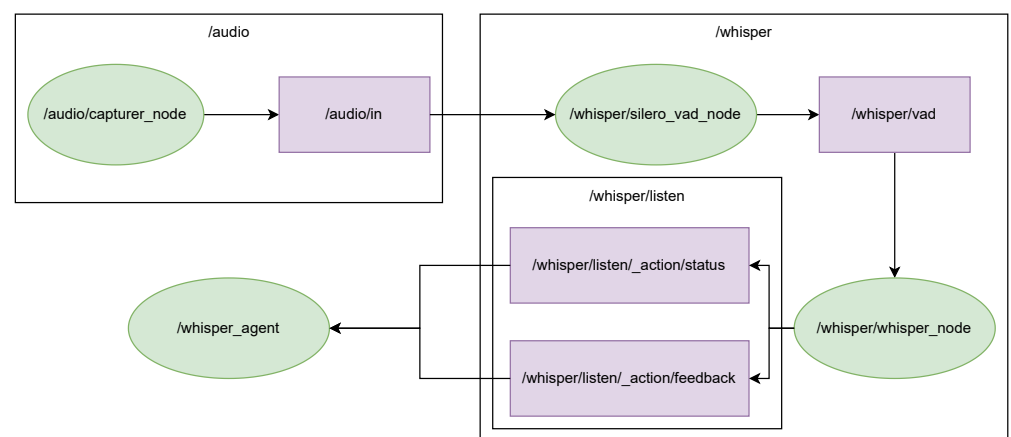


Figure 9. ROS 2 speech-to-text pipeline.

This pipeline has been developed using a custom version of the ROS 2 stack “whisper_ros” (https://github.com/mgonzs13/whisper_ros, accessed on 25 June 2025). This stack provides the foundational capabilities for integrating the speech to text into ROS 2. It is based on the whisper.cpp (version 1.7.6) (<https://github.com/ggerganov/whisper.cpp>, accessed on 25 June 2025), a high-performance inference of OpenAI’s Whisper automatic speech recognition and on the PortAudio (version 19.7.0) (<https://www.portaudio.com/>, accessed on 25 June 2025) library to record the audio.

The first step is to record the audio through the */audio_capturer_node*. This step is carried out by the PortAudio library, which is a free, cross-platform, open-source, audio I/O library that provides a very simple API for recording and/or playing sound using a simple callback function or a blocking read/write interface. The voice activity detector then uses this audio to transcribe a short speech segment, which is carried out by the */whisper/silero_vad_node*. This node is built on the SileroVAD (version 5.0) (<https://github.com/snakers4/silero-vad>, accessed on 25 June 2025) model, which is a machine learning model designed to detect speech segments. It is lightweight, at just 1.8 MB, and is capable of processing 30 ms chunks in approximately 1 ms, making it both fast and highly accurate. Finally, the last step consists of the transcription of the detected speech segment (published by the */whisper/vad* topic) through the */whisper_node*. Whisper (model ggml-large-v3-turbo-q5_0.bin) (<https://openai.com/index/whisper/>, accessed on 25 June 2025) is an automatic speech recognition (ASR) system trained on 680,000 h of multilingual and multitask supervised data collected from the web. The use of such a large and diverse dataset leads to improved robustness to accents, background noise, and technical language. Moreover, it enables transcription in multiple languages, as well as translation from those languages into English. The final transcribed text is published and read by the agents connected to the DSR. The STT agent uploads the final text transcribed into the DSR graph as an attribute of an action node called “listen” that is connected to the robot. This attribute is saved in the internal blackboard of the Behavioral Tree as an output, so the next node could access that information to build the explanation.

5.2. Natural Language Processing Agent

Once the question has been transcribed, the next step is to identify the robot event associated with the question and the role of the questioner [14]. These tasks are addressed by the Natural Language Processing (NLP) agent. The output is a raw causal explanation, composed of the description of the identified event and its causes detailed in the high-level episodic memory (causal log).

5.2.1. Event Recognition

In this step, the transcribed question is mapped to a specific robot event. Rasa’s intent classification system (<https://rasa.com/docs/>, accessed on 27 June 2025) was employed. While it is provided as a means to learn semantic intents of utterances, we utilized it to learn events by training it with 20 questions per event. For example, the question “Why did you stop following me?” was associated with the event “Robot.task_abort = following”.

5.2.2. Role Recognition

In the same fashion, as for event recognition, we leveraged Rasa’s intent recognition framework to predict the social role of the person asking the question [14]. Specifically, we considered four distinct roles: therapist, technician, resident, and familiar.

Social role shapes human interaction by influencing not only the subject matter of communication but also the manner in which messages are conveyed. In the context of generating explanations, it is reasonable to hypothesize that a person’s social role affects both the content and phrasing of their questions. Consequently, a robot capable of producing effective, context-sensitive explanations should be aware of the human’s social role. To enable this capability, we trained a model to map human queries to corresponding social roles. The model was developed using a dataset of 160 manually labeled examples, with 40 representative queries assigned to each of the four roles. For instance, the social role associated with the question “Why was the robot interacting with my father?” will be *familiar*.

5.2.3. Cause–Effect Extraction

Finally, this step is responsible for searching for the most recent occurrence of an effect that matches the recognized event in the causal log file. The descriptions of the associated causes are then extracted from the semantic memory, and constitute what we denote the “raw explanation”. Some examples are presented in Section 6.2.2.

5.2.4. Prompt Generation

Once we have the “raw explanation”, the LLaMA model prompt is constructed based on the question, the recognized role, and the raw explanation. The final result is the following prompt: “You are a social assistive robot. According to the role [*role*] of the target that you are answering and the question asked [*question*]. Compress the answer with the meaningful information: [*raw explanation*]”.

5.3. LLaMA Agent

Although the raw explanations generated by the system contain accurate and relevant information, their formulation may lack clarity or naturalness, potentially hindering the human’s understanding. To address this, the LLaMA software agent performs a final refinement step using a Large Language Model (LLM) to improve fluency, personalization, and semantic quality [14]. The goal is to generate a final explanation, answering the question asked by the human (see Figure 8). The role-based personalization is embedded in the model instruction, but the model also has his own capabilities to give a personalized explanation with its knowledge. The following instruction is used: “You are an assistance robot interacting with different types of people in a residential facility. The roles of the people you interact with are therapist, technician, and resident. Responses should be tailored to each role, with the most technical, direct, and detailed communication reserved for the technician; collaborative and empathetic interaction with the therapist; and clear, accessible, and friendly language with the resident.”

When the agent receives the notification, it invokes `/llama_node` to generate a more natural, human-friendly, and contextually appropriate response for the previously detected user role. As shown in the ROS 2 pipeline in Figure 10, the final output is published through the main topic `/llama/generate_generate_response`, which the LLaMA agent monitors. This agent then uploads the refined explanation as an attribute to the corresponding node in the DSR, allowing the output agents (TTS and Web agents, responsible for managing the speech and screen interfaces, respectively) to present the explanation clearly and effectively to the human. In our experiments, we used the Meta-Llama-3.1-8B-Instruct-Q4_k_M.gguf model, a quantized 8-billion-parameter version fine-tuned for conversational and instructional tasks. Its ability to run locally supports real-time, on-device interaction, making it suitable for social robotics applications.

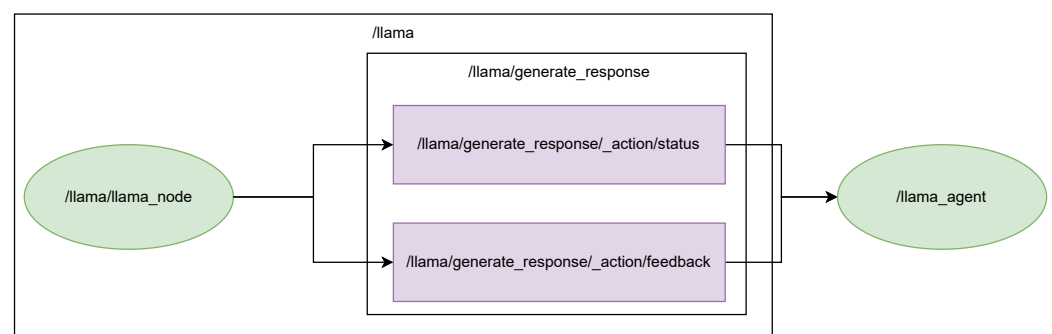


Figure 10. ROS 2 final response pipeline.

5.4. Text To Speech Agent

Piper (version 1.2.0) (<https://github.com/rhasspy/piper>, accessed on 27 June 2025) is a collection of high-quality, open-source text-to-speech voices developed by the Piper Project, powered by machine learning technology. These voices are synthesized in browser, requiring no cloud subscriptions, and are entirely free to use. You can use them to read aloud web pages and documents with the Read Aloud extension, or make them generally available to all browser apps through the Piper TTS extension.

Each of the voice packs is a machine learning model capable of synthesizing one or more distinct voices. Each pack must be separately installed. Due to the substantial size of these voice packs, it is advisable to install only those that you intend to use. To assist in your selection, you can refer to the “Popularity” ranking, which indicates the preferred choices among users.

This package is integrated into ROS 2 using the PortAudio library to play the audio. The process is based on the pipeline developed in Figure 11.

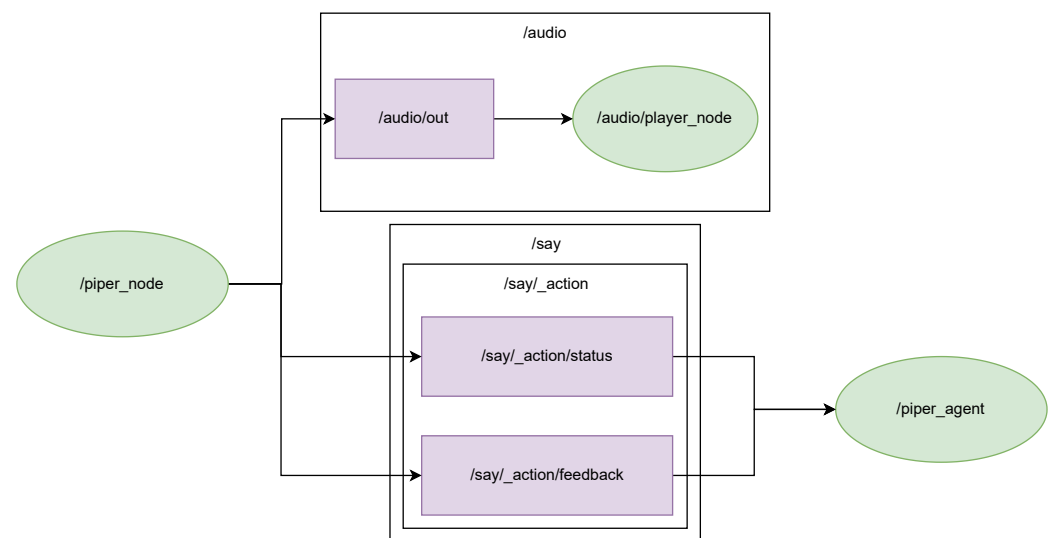


Figure 11. ROS 2 Text to Speech pipeline.

In real-world scenarios, providing information through voice offers several important advantages. It is particularly beneficial for individuals with visual impairments, as it allows them to access information without needing to rely on reading text. Voice communication also provides a hands-free experience, making it easier for humans to interact with the system while performing other tasks. Additionally, voice allows for a more natural and conversational interaction, which can improve the human experience by making the system feel more intuitive and human-like. Furthermore, spoken information can be processed more quickly than reading text, especially when the human is on the move or engaged in other activities. Finally, voice can create a more personal connection, making interactions feel more engaging and dynamic. Overall, voice-based communication enhances accessibility, convenience, and the overall user experience.

5.5. Web Agent

The Web agent provides a bridge to link the robot’s on-screen graphical user interfaces (GUIs) with the DSR. Specifically, this agent uses the WebSockets (version september 2023) (<https://websockets.spec.whatwg.org/>, accessed on 20 June 2025) protocol to establish real-time, bidirectional communication over a single TCP connection. WebSockets support asynchronous data exchange, enabling the client (typically a web browser) and the server to

independently send and receive messages in parallel. The agent is implemented in C/C++ and operates in two principal modes: receiving input and transmitting output, as follows:

- **Receiving from DSR:** To ensure that the GUI reflects the robot's internal state (such as the current task status), the agent monitors specific changes in the DSR. When such updates occur, the agent triggers corresponding modifications on the interface. These changes can range from subtle updates to images or text to complete layout changes on either the robot's own screen or on an external display.
- **Sending to DSR:** Through interactive elements on the GUI (like touchscreen buttons or wireless keypad controls), human users can provide input such as responses or preferences. The agent translates this interaction into updates within the DSR, which may involve modifying personal attribute data (e.g., user emotional state) or initiating new robot behaviors by triggering predefined external use cases.

In this architecture, this agent is responsible for displaying the question and the final answer on the robot's screen, which is essential in robot–human interactions. Providing information through text on a screen offers several key advantages. First, it ensures accessibility for users with hearing impairments, allowing them to receive information without relying on audio. Additionally, text helps to clarify and reinforce the message, especially in noisy environments or situations where audio may not be clearly heard. Offering text also improves comprehension, as some humans process information better when they read it rather than hear it. Furthermore, it enhances usability in various contexts, as users can refer back to the text at their own pace.

6. Experimental Results

6.1. Experimental Setting

The Ambient Assisted Living (AAL) system has been installed in a small three-room apartment within our research facility. The apartment includes a kitchen and dining area, a bathroom, and a bedroom. Figure 12 illustrates the apartment layout. Two ceiling-mounted panoramic cameras with 360-degree coverage (installed at approximately 2.4 m in height) are employed to track occupant movement in the bedroom and dining area. Vital signs, such as heart rate and respiration, are monitored using a 60 GHz frequency modulated continuous wave (FMCW) radar unit [23]. This unit is positioned under the bed for nighttime monitoring and behind an armchair in the dining area for daytime assessment. The system also integrates magnetic contact sensors on doors and windows to detect open/closed states, an environmental sensor to measure parameters such as temperature and humidity, and a presence detector in the bathroom.

The robotic platform used in these experiments is a Morphia unit, developed by MetraLabs GmbH (Ilmenau, Germany) in the framework of the joint research project MORPHIA (Mobile Robotic Care Assistant to Improve Participation, Care, and Safety in Home Care through Video-based Relatives Network) [24]. Built on the TORY differential drive base, it features a full-perimeter circular bumper for physical safety and a SICK s300 (SICK AG, Waldkirch, Germany) laser rangefinder for obstacle detection and mapping. Navigation capabilities are enhanced by an Intel RealSense D435i RGB-D camera (Intel, Santa Clara, CA, USA), while environmental perception is supported by three 2MP cameras (Valeo, Paris, France) and an Azure Kinect RGB-D sensor (Microsoft, Redmond, WA, USA). The Azure Kinect also has a seven-microphone array for capturing far-field sounds and speech, useful for integrating speech recognition. The system integrates a tablet interface and onboard audio via speakers for user interaction. Figure 13 illustrates the robot operating within the experimental apartment. All the processing and control are handled by a dual-computing setup comprising an Intel NUC i7 (Intel, Santa Clara, CA, USA) and an NVIDIA Jetson Orin AGX (Nvidia, Santa Clara, CA, USA).

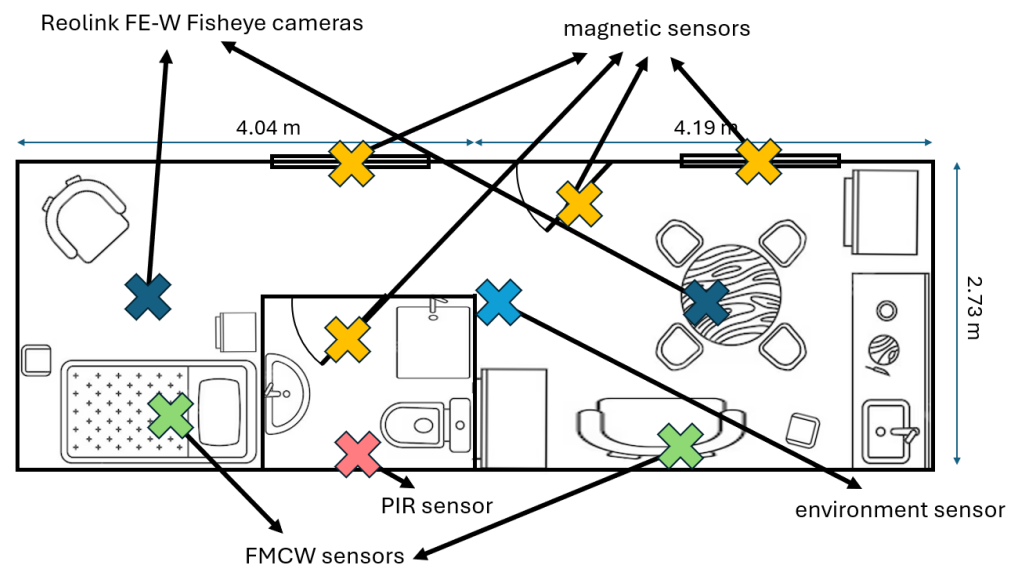


Figure 12. Layout of the small apartment and the distribution of sensors.



Figure 13. The Morphia robot navigating in the small apartment.

6.2. Example of Use Case

To demonstrate the functionality of the developed software, we present a step-by-step execution example based on the *Following* task. This includes the evolution of the working memory (DSR) and the corresponding growth of the causal log over time. In addition, we present several example explanations generated from the causal log in response to different user queries, illustrating how the system adapts its responses based on context.

6.2.1. DSR Evolution and Causal Log Construction

To simplify the figures, DSR snapshots are unified in some instances with several representative changes. Changes between snapshots are drawn in yellow. The table in the figures represents the causal log file, with events on the left and causes on the right. This table is simplified by cutting the timestamp and the index column.

Figure 14 shows the DSR in an initial state, where the robot is waiting and exploring opportunities to perform a task. In this case, the causal log starts with the robot waking up and initial information such as the state of the tray and the robot's first task.

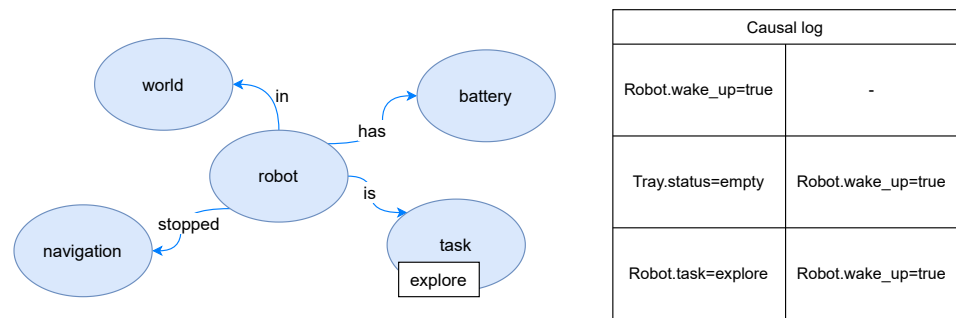


Figure 14. (Left) A snapshot of the DSR where the robot is exploring and (right) first rows of the causal log.

At a given moment, the person detection agent running inside CORTEX detects a person identified as a “therapist”. At the same time, the infrared sensor on the robot’s tray returns true, so by logic, the therapist has placed the medicine on the tray. Finally, as the robot detects a known person, the new task is *Interaction*. This means that the robot is ready to listen to commands to execute the requested task. This whole new scenario is shown in Figure 15. In the causal log of the figure, you can see how each DSR event is annotated along with its causes.

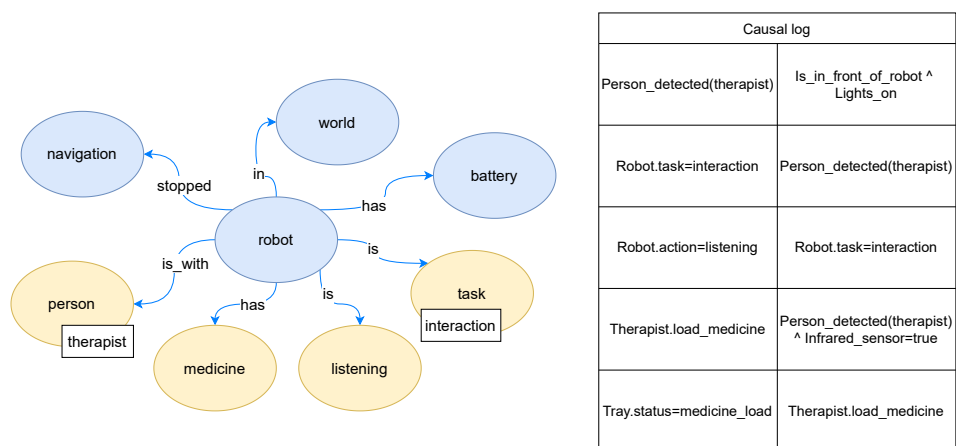


Figure 15. (Left) The DSR is changed when the therapist is detected and the medicine loaded on tray, and (right) events and causes associated for this change.

As indicated above, the robot is in *Interaction* mode waiting for commands from the therapist. In this case, the command is “follow”, so the robot, understanding this command, changes its task to *Following* and starts the action of moving. These changes are depicted in Figure 16.

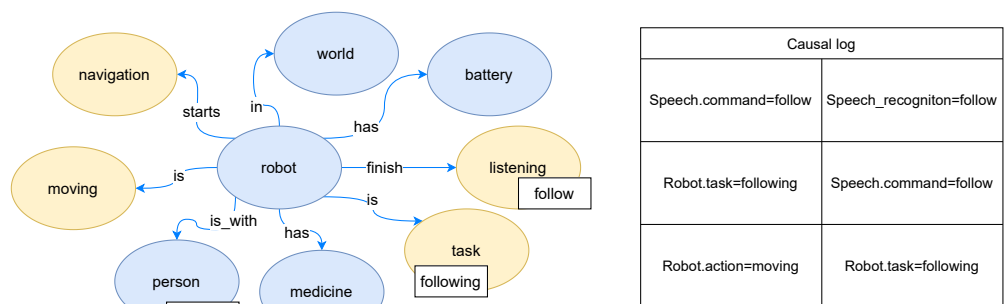


Figure 16. (Left) DSR when the robot starts following the therapist, and (right) causal log annotation of the command and actions.

During the following, the robot's battery started to run low, and in addition, the robot started to detect some humans around it and along the path (three people to be precise). Based on this information, the QoSAgent decided to abort the task, as it deemed the path unsafe, and given the battery level, completion of the task would be difficult. At that moment, the task was aborted, and all related actions were stopped as well. To alert the therapist, the *Alarm* task is performed with a short speech stating that it cannot continue the follow-up. This whole evolution is briefly shown in Figure 17.

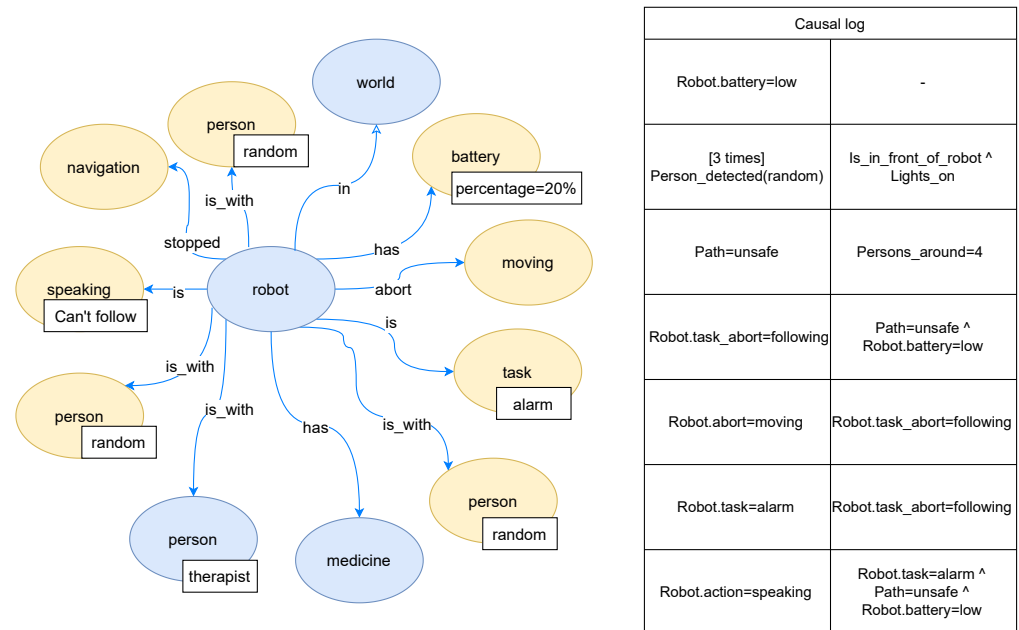


Figure 17. (Left) The DSR shows all the anomalies that cause the abort of the following, and (right) the evolution of the causal log for this case.

Finally, at this point in the DSR evolution, the robot returns to *Interaction* mode because the therapist is continuously close to it and can ask for an explanation of what happens during the next task, as can be seen in Figure 18.

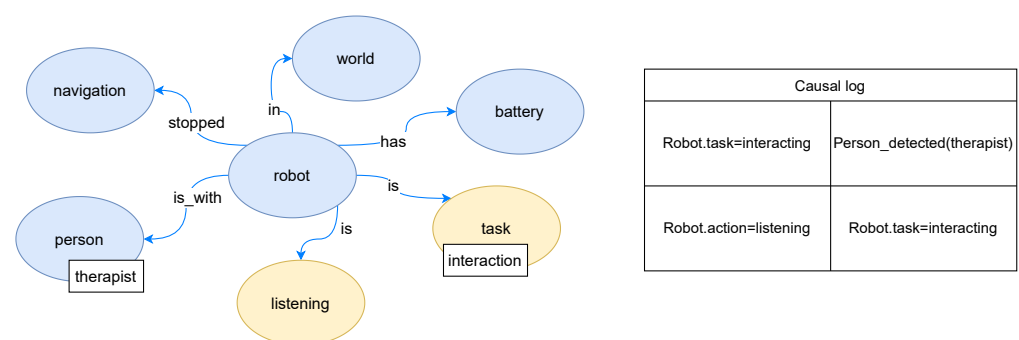


Figure 18. (Left) The robot returns to *Interaction* mode, and (right) causes of this event.

Figure 19 represents that the therapist had said the corresponding command “explain”, and the robot started to execute this task, being the first step listening to the question that the therapist should ask.

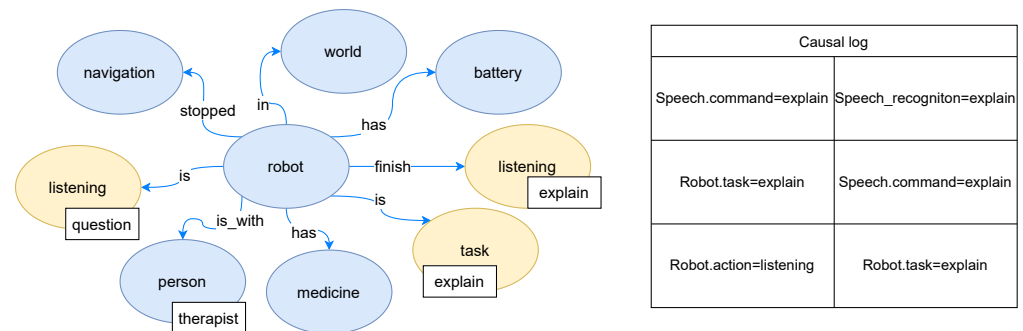


Figure 19. (Left) Final snapshot when the therapist asks for an explanation, and (right) command and actions to start performing the explanation.

6.2.2. Examples of Explanation Generated

To illustrate the system's performance, a few examples have been detailed in this section. Each response is tailored to the human's query, considering both context factors and the human's social role. Additionally, for better comparison, the raw explanation without syntactical refinement and role adaptation is shown in the examples. In all queries, the role detected is therapist.

Example 1

Question: "Why did you stop following me"?

Raw answer: "Cause: The path was unsafe because there were more than four persons in robot field of view and in the middle of the path. The robot's battery is running low. Effect: The robot stopped following mode."

Answer: I stopped following you because there were too many people in the path ahead, making it unsafe for me to continue. Additionally, my battery level is low, so I had to stop to prevent any potential risks.

Example 2

Question: "Why did you start following me"?

Raw answer: "Cause: The therapist commanded me to start following. The therapist was detected by the robot's camera. Effect: The robot started the use case for following the therapist."

Answer: Because you, the therapist, instructed me to start following.

Example 3

Question: Why was the path unsafe?

Raw answer: "Cause: There were three persons in the robot field of view. Effect: The path was unsafe because there were more than four persons in the robot field of view and in the middle of the path."

Answer: The path was unsafe because there were three people in my field of view, exceeding the safety threshold of people around.

6.2.3. Deployment in the Morphia Robot

To validate the system in a real scenario, the *Following* task was executed on the Morphia robot. As shown in Figure 20, the robot successfully followed the user and responded to a spoken explanation request, showcasing its causal reasoning and interaction capabilities in a physical environment.



Figure 20. Use case in Morphia robot: (left) following behavior, and (right) explanation generated to a question.

7. Discussion

In this section, we analyze the system’s evaluation results, differentiating between a qualitative and quantitative evaluation and addressing key limitations such as error handling and generalizability for unseen scenarios.

7.1. Evaluation Results

7.1.1. Quality of the Answer

To evaluate the explanations generated, an experiment was conducted with 30 participants. The three questions and explanations above were presented to the participants. The participants then filled out an online questionnaire with five qualitative Likert items for clarity, completeness, terminological accuracy, and linguistic adequacy of the explanations.

The age distribution of the participants was under 30 years (20%), 30–44 years (16.67%), 45–60 years (43.33%), and over 60 years (20%). Of the participants 56.67% were male and 43.33% were female. The results are presented in Figure 21 (5 is the highest score, and 1 is the lowest score).

The explanations scored generally high marks in content, clarity, and spelling, although occasional failures in terminological accuracy were identified. It is concluded that the system is adequate to generate appropriate answers for the questions but possibly requires some improvement in the use of technical terminology.

7.1.2. Latency and Resources

With an NVIDIA GeForce RTX 5060 Ti (Nvidia, Santa Clara, CA, USA), the transcription time for the Whisper speech recognition module for the question “Why did you stop following me?” is 0.469 s. The response time of the LLaMA agent to generate the answer to the question is 0.795 s. When running on the robot, using the NVIDIA Jetson AGX Orin (Nvidia, Santa Clara, CA, USA), the transcription time for the same question is 1.861 s and the response time to generate the answer is 3.645 s. When also the perception software is launched, the times are lightly higher: 2.144 s and 4.410 s, respectively.

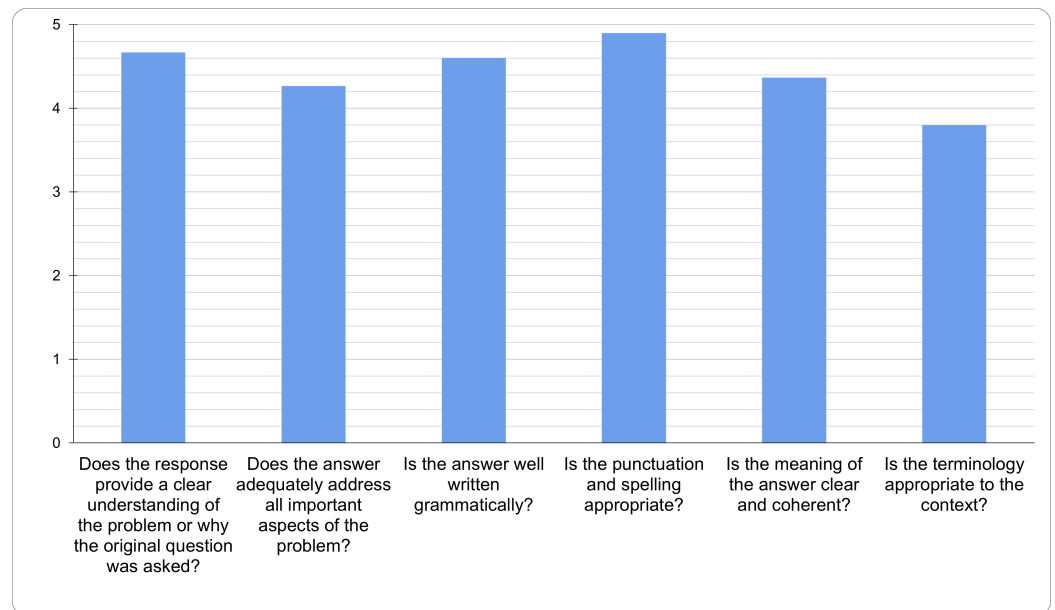


Figure 21. Qualitative evaluation of robot-generated responses.

The maximum operation temperature is 42.25 °C, and the GPU shared RAM usage is 5.6 GB of the available 61.4 GB. At the moment of execution, the total GPU load is 99%, but only for a few seconds. These values are obtained during the LLaMA execution and are presented in Figure 22.

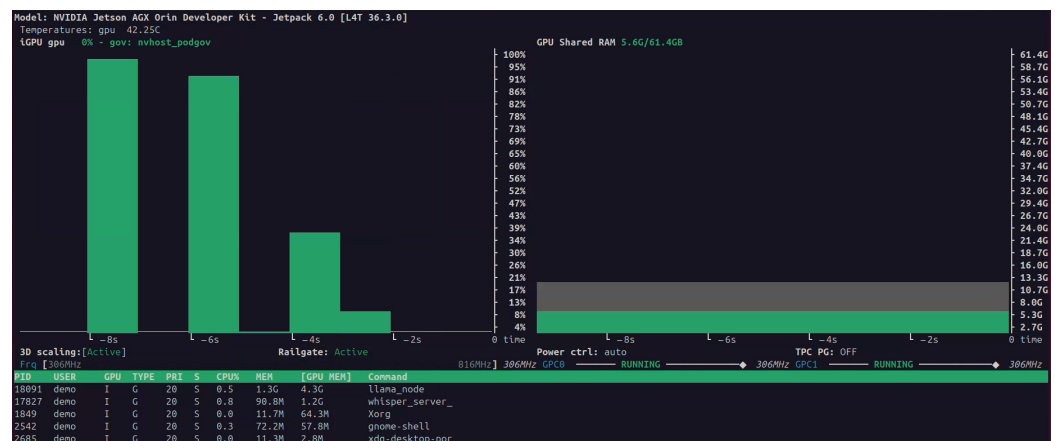


Figure 22. NVIDIA Jetson AGX Orin (Nvidia, Santa Clara, CA, USA) Resources during the Llama node execution.

7.1.3. Analysis of Results

The experimental results demonstrate that the system successfully captures relevant events from its runtime knowledge model, constructs causal chains of events (causal log), and generates personalized explanations using Natural Language Processing agents and LLM refinement. In particular, the use of a distributed architecture in which agents interact via shared memory rather than direct messaging improves modularity, scalability, and robustness in dynamic environments.

7.2. Error Handling and Robustness

The current system assumes accuracy in speech recognition and explanation. However, real-world deployments require robust mechanisms to address various sources of error. In the following, we propose handling strategies for future research:

- **Speech understanding failures:** If a user requests an action that is not part of the robot's predefined task set, the system should respond by informing the user that it cannot perform the requested action and suggests rephrasing or choosing another task. In cases where a misunderstood question leads to an incoherent explanation, the user is encouraged to simply repeat the question for clarification.
- **Explanation errors:** If a generated explanation is incorrect or incoherent, user trust may be degraded. A long-term solution could involve applying reinforcement learning with human preferences to optimize explanation policies based on user feedback. Recent approaches [25] show that models can be tuned to prefer outputs rated more helpful or truthful by users. Incorporating similar reward models into the explanation refinement pipeline could help the robot learn which types of explanations better align with human expectations and feedback over time.

7.3. Limitation for Unseen Scenarios

One limitation of the current implementation is that the causal representation depends on predefined design time events (e.g., preconditions in Behavior Trees), which restrict the system's generalizability to unknown tasks and to any question out of its knowledge. Although the system was validated in a real environment using the Morphia robot, broader evaluations in less structured or more complex settings would help assess its robustness under greater variability. To improve generalizability, future work could explore the following:

- **Prompting Strategies for Multi-Path Reasoning:** Modern prompting techniques can be used to explore multiple reasoning paths within LLM, especially when ambiguity or novelty is detected in user questions. For instance, self-ask prompting or chain-of-thought prompting can lead the LLM to internally generate intermediate reasoning steps before providing a final explanation [26]. This increases the robustness of the system in cases where predefined causal chains are incomplete.
- **Retrieval-Augmented Explanation Generation:** Integrating retrieval-augmented generation allows the robot to dynamically incorporate external knowledge or memory snippets during explanation synthesis [27]. The system could query past episodic logs or even external structured sources (e.g., semantic knowledge bases) to enrich responses when internal memory is insufficient.

These strategies would significantly improve the robot's capacity to adapt, reason flexibly, and deliver coherent explanations in complex, changing, or partially unknown environments.

8. Conclusions

This work presents an integrated framework that enables a social robot to generate coherent and contextually appropriate explanations of its own behavior based on a high-level episodic memory and an agent-based software architecture. Unlike previous approaches that rely on low-level log data or purely statistical models, our system leverages semantically and causally structured representations, allowing it to produce meaningful explanations even after extended periods of interaction.

The system has proven capable of identifying relevant events, maintaining compact but informative long-term memory, and generating appropriate linguistic responses within reasonable latency limits, even when running on edge computing devices. These features make the proposed framework suitable for real-world applications in socially assistive robotics.

Future work could explore the dynamic expansion of the causal repertoire using learning techniques or the inclusion of affective and theory-of-mind elements to further enrich

explanation personalization and social interaction capabilities. Another promising direction is the incorporation of reinforcement learning based on human preferences, allowing the robot to improve the quality and relevance of its explanations over time. Finally, we intend to explore retrieval-augmented generation techniques to dynamically incorporate external knowledge sources when internal memory is insufficient, thus increasing robustness in dynamic and unknown environments.

Author Contributions: J.G.: conceptualization, methodology, software, validation, formal analysis, investigation, writing—original draft preparation, writing—review and editing, visualization; A.T.: conceptualization, software, formal analysis, investigation; Ó.P.: conceptualization, software, formal analysis, investigation; S.B.: conceptualization, methodology, validation, formal analysis, resources, writing—review and editing, visualization, supervision, project administration, funding acquisition; T.H.: conceptualization, methodology, validation, formal analysis, resources, writing—review and editing, visualization, supervision, project administration, funding acquisition; A.B.: conceptualization, methodology, validation, formal analysis, resources, writing—original draft preparation, writing—review and editing, visualization, supervision, project administration, funding acquisition. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the grant numbers PDC2022-133597-C4X, TED2021-131739B-C2X, and PID2022-137344OB-C3X, funded by MCIN/AEI/10.13039/501100011033 and by the European Union NextGenerationEU/PRTR (for the first two grants) and “ERDF A way of making Europe” (for the third grant). This research was partly funded by the Swedish Research Council Vetenskapsrådet through grant number 2022-04674.

Data Availability Statement: The semantic definition in memory and an example of a causal log can be found in this Google Drive link: https://drive.google.com/drive/folders/1Nq3koVohe_MrEEYAZkXXDnsmEUDcvcv?usp=sharing, accessed on 4 July 2025. Furthermore, the questionnaire given to the participants can be found in this corresponding Google Form link: <https://forms.gle/cVNrGtdnggho5zLkn9>, accessed on 4 July 2025.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Sobrin-Hidalgo, D.; Manuel Guerrero-Higueras, A.; Matellan-Olivera, V. Generating Explanations for Autonomous Robots: A Systematic Review. *IEEE Access* **2025**, *13*, 20413–20426. [\[CrossRef\]](#)
2. Sakai, T.; Nagai, T. Explainable autonomous robots: A survey and perspective. *Adv. Robot.* **2022**, *36*, 219–238. [\[CrossRef\]](#)
3. Campagna, G.; Rehm, M. A Systematic Review of Trust Assessments in Human–Robot Interaction. *J. Hum.-Robot Interact.* **2025**, *14*, 30. [\[CrossRef\]](#)
4. Peller-Konrad, F.; Kartmann, R.; Dreher, C.R.; Meixner, A.; Reister, F.; Grotz, M.; Asfour, T. A memory system of a robot cognitive architecture and its implementation in ArmarX. *Robot. Auton. Syst.* **2023**, *164*, 104415. [\[CrossRef\]](#)
5. Stange, S.; Hassan, T.; Schröder, F.; Konkol, J.; Kopp, S. Self-Explaining Social Robots: An Explainable Behavior Generation Architecture for Human-Robot Interaction. *Front. Artif. Intell.* **2022**, *5*, 866920. [\[CrossRef\]](#) [\[PubMed\]](#)
6. Chakraborti, T.; Sreedharan, S.; Zhang, Y.; Kambhampati, S. Plan explanations as model reconciliation: Moving beyond explanation as soliloquy. In Proceedings of the 26th International Joint Conference on Artificial Intelligence, Melbourne, VIC, Australia, 19–25 August 2017; pp. 156–163.
7. Bärman, L.; DeChant, C.; Plewnia, J.; Peller-Konrad, F.; Bauer, D.; Asfour, T.; Waibel, A. Episodic Memory Verbalization using Hierarchical Representations of Life-Long Robot Experience. *arXiv* **2024**, arXiv:2409.17702. [\[CrossRef\]](#)
8. Rosenthal, S.; Selvaraj, S.P.; Veloso, M. Verbalization: Narration of autonomous robot experience. In Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, New York, NY, USA, 9–15 July 2016; pp. 862–868.
9. Flores, J.G.; Meza, I.; Colin, É.; Gardent, C.; Gangemi, A.; Pineda, L.A. Robot experience stories: First person generation of robotic task narratives in SitLog1. *J. Intell. Fuzzy Syst.* **2018**, *34*, 3291–3300. [\[CrossRef\]](#)
10. Bärman, L.; Peller-Konrad, F.; Constantin, S.; Asfour, T.; Waibel, A. Deep Episodic Memory for Verbalization of Robot Experience. *IEEE Robot. Autom. Lett.* **2021**, *6*, 5808–5815. [\[CrossRef\]](#)
11. Minaee, S.; Mikolov, T.; Nikzad, N.; Chenaghlu, M.A.; Socher, R.; Amatriain, X.; Gao, J. Large Language Models: A Survey. *arXiv* **2024**, arXiv:2402.06196. [\[PubMed\]](#)

12. Zhou, Y.; Liu, Z.; Dou, Z. How Credible Is an Answer From Retrieval-Augmented LLMs? Investigation and Evaluation with Multi-Hop QA. In Proceedings of the 31st International Conference on Computational Linguistics, Abu Dhabi, United Arab Emirates, 19–24 January 2025; pp. 4232–4242.
13. Marfil, R.; Romero-Garcés, A.; Bandera, J.; Manso, L.; Calderita, L.; Bustos, P.; Bandera, A.; Garcia-Polo, J.; Fernandez, F.; Voilmy, D. Perceptions or Actions? Grounding How Agents Interact Within a Software Architecture for Cognitive Robotics. *Cogn. Comput.* **2019**, *12*, 479–497. [\[CrossRef\]](#)
14. Galeas, J.; Bensch, S.; Hellström, T.; Bandera, A. Personalized causal explanations of a robot’s behavior. *Front. Robot. AI* **2025**, *under review*.
15. Galeas, J.; Tudela, A.; Pons, O.; Bandera, J.P.; Bandera, A. Design of a Cyber-Physical System-of-Systems Architecture for Elderly Care at Home. *Electronics* **2024**, *13*, 4583. [\[CrossRef\]](#)
16. Chakraborti, T.; Sreedharan, S.; Kambhampati, S. The Emerging Landscape of Explainable Automated Planning & Decision Making. In Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, Yokohama, Japan, 11–17 July 2021.
17. Lindner, F.; Olz, C. Step-by-Step Task Plan Explanations Beyond Causal Links. In Proceedings of the 2022 31st IEEE International Conference on Robot and Human Interactive Communication (RO-MAN), Napoli, Italy, 29 August–2 September 2022; pp. 45–51. [\[CrossRef\]](#)
18. Das, D.; Banerjee, S.; Chernova, S. Explainable AI for Robot Failures: Generating Explanations That Improve User Assistance in Fault Recovery. In Proceedings of the 2021 ACM/IEEE International Conference on Human-Robot Interaction, Boulder, CO, USA, 8–11 March 2021; pp. 351–360. [\[CrossRef\]](#)
19. Stulp, F.; Bauer, A.S.; Bustamante Gomez, S.; Lay, F.S.; Schmaus, P.; Leidner, D. Explainability and Knowledge Representation in Robotics: The Green Button Challenge. In Proceedings of the Explainable Logic-Based Knowledge Representation (XLoKR 2020) at the 17th International Conference on Principles of Knowledge Representation and Reasoning (KR 2020), Rhodes, Greece, 12–18 September 2020.
20. Canal, G.; Krivic, S.; Luff, P.; Coles, A. Task Plan verbalizations with causal justifications. In Proceedings of the ICAPS 2021 Workshop on Explainable AI Planning (XAIP), Guangzhou, China, 2–13 August 2021.
21. Canal, G.; Krivic, S.; Luff, P.; Coles, A. PlanVerb: Domain-Independent Verbalization and Summary of Task Plans. *Proc. AAAI Conf. Artif. Intell.* **2022**, *36*, 9698–9706. [\[CrossRef\]](#)
22. Romero-Garcés, A.; Hidalgo-Paniagua, A.; Bustos, P.; Marfil, R.; Bandera, A. Inner Speech: A Mechanism for Self-coordinating Decision Making Processes in Robotics. In *ROBOT2022: Fifth Iberian Robotics Conference*; Tardioli, D., Matellán, V., Heredia, G., Silva, M.F., Marques, L., Eds.; Springer: Cham, Switzerland, 2023; pp. 588–599.
23. Alizadeh, M.; Shaker, G.; Almeida, J.C.M.D.; Morita, P.P.; Safavi-Naeini, S. Remote Monitoring of Human Vital Signs Using mm-Wave FMCW Radar. *IEEE Access* **2019**, *7*, 54958–54968. [\[CrossRef\]](#)
24. Wengefeld, T.; Schuetz, B.; Girdziunaite, G.; Scheidig, A.; Gross, H.M. The MORPHIA Project: First Results of a Long-Term User Study in an Elderly Care Scenario from Robotic Point of View. In Proceedings of the ISR Europe 2022, 54th International Symposium on Robotics, Munich, Germany, 20–21 June 2022; pp. 1–8.
25. DeepSeek-AI; Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv* **2025**, arXiv:2501.12948.
26. Kojima, T.; Gu, S.S.; Reid, M.; Matsuo, Y.; Iwasawa, Y. Large Language Models are Zero-Shot Reasoners. *arXiv* **2023**, arXiv:2205.11916. [\[CrossRef\]](#)
27. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv* **2021**, arXiv:2005.11401.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.